Biweekly Assessment BW06 — Weeks 11–12

Sessions covered: S51–60
Topics: PK/FK modeling; INNER JOIN; LEFT JOIN + multi-table; TCL + indexes lite; JDBC `Connection` + try-with-resources; `PreparedStatement` SELECT; JDBC INSERT / UPDATE / DELETE (`?` only); library CRUD + SQL injection; HTML structure & semantic; forms & a11y basics

Timebox: 45–60 minutes (homework or Friday of the even week).
Rules: Closed notes unless your trainer allows open notes. No copying.
Coding constraint: JOIN SQL and JDBC (`Connection` / `PreparedStatement` / try-with-resources) are OK. **`PreparedStatement` `?` only** — no `java.sql.Statement` and no string-concat SQL. Password from env / `.env` — never a literal in source. **No Spring** / JPA / Hibernate / Lombok / `DataSource` / Hikari / `application.yml`. HTML structure + forms (labels, a11y) OK. **No CSS** / Flex / media queries / JS / DOM / `fetch`. No React.

A. Multiple Choice (12)

Circle the best option.

1. A FOREIGN KEY on `loans.member_id` mainly:
   a) Points at one PRIMARY KEY value in `members` so a loan belongs to one member
   b) Replaces `COMMIT`
   c) Is a CSS class
   d) Opens Spring Boot
2. `SELECT m.name, b.title FROM members m INNER JOIN loans l ON m.member_id = l.member_id INNER JOIN books b ON b.book_id = l.book_id` returns:
   a) Every member, even with no loans
   b) Only rows where the JOIN conditions match (members who have a loan to a book)
   c) A Git commit
   d) HTML `<form>` fields
3. To list **every TechMart** canteen stall, including stalls with **no** orders yet:
   a) INNER JOIN stalls to orders only
   b) `DELETE FROM stalls`
   c) `LEFT JOIN` stalls to orders (unmatched order columns are NULL)
   d) `ROLLBACK` twice
4. After `START TRANSACTION` and two `UPDATE`s, `ROLLBACK`:
   a) Makes both UPDATEs permanent
   b) Undoes uncommitted work since the transaction started
   c) Creates an HTML footer
   d) Registers Connector/J
5. `CREATE INDEX idx_books_author ON books (author)` mainly:
   a) Speeds `WHERE` / `JOIN` / `ORDER BY` on `author`; writes pay a small extra cost
   b) Replaces PRIMARY KEY
   c) Is `application.yml`
   d) Injects SQL
6. `try (Connection conn = DriverManager.getConnection(url, user, password)) { ... }` mainly:
   a) Auto-closes the `Connection` (`AutoCloseable`) even if `SQLException` is thrown
   b) Needs Spring `DataSource` / Hikari

c) Commits git
d) Skips the driver JAR at run time

7. For `SELECT title FROM books WHERE author = ?`, placeholders are filled with:
   a) `"'" + author + "'"` glued into the SQL string
   b) `ps.setString(1, author)` — JDBC `?` indexes are **1-based**
   c) Lombok
   d) A `<div>`
8. `INSERT INTO canteen_items (name, price_inr, stall) VALUES (?, ?, ?)` should run with:
   a) `executeQuery` only
   b) `Class.forName` instead of SQL
   c) `executeUpdate` (returns the row count)
   d) `git push`
9. `Statement` + `"WHERE title = '" + title + "'"` is banned on the core path because:
   a) A typed title can change the command (SQL injection), e.g. `' OR '1'='1`
   b) MySQL cannot store VARCHAR
   c) It is the only way to SELECT
   d) It replaces `<label>`
10. The MySQL password in JDBC homework should come from:
    a) A string literal in `LibraryCrud.java` committed to GitHub
    b) Env (`MYSQL_PASSWORD`) or a local `.env` (commit `.env.example` only)
    c) `application.yml` (that file is Spring — not this window)
    d) An HTML `id`
11. The semantic tag for the page's unique main content is:
    a) `<style>`
    b) `<div>` only, forever
    c) `<main>`
    d) `<script>`
12. A CampusCart / **TechMart** UPI / order form should:
    a) Use `<label for="...">` matching each input `id`, and `method="post"` so payment data is not stuck in the URL
    b) Skip labels; use GET for the UPI PIN
    c) Need Flexbox and media queries
    d) Call `fetch()` / Spring

B. Coding (2)

1. Write `CanteenStock`. Assume `training_db` already has:

`canteen_items (item_id INT PK AUTO_INCREMENT, name VARCHAR(80), price_inr INT, stall VARCHAR(40))`

Password from `System.getenv("MYSQL_PASSWORD")` (or a local `.env`) — if missing, print `Set MYSQL_PASSWORD first` and return. **Never** a password literal. JDBC URL to `localhost` / `training_db` (lab flags OK). try-with-resources on `Connection` and `PreparedStatement`.

- `listByStall(conn, stall)` — `SELECT name, price_inr FROM canteen_items WHERE stall = ? ORDER BY name` — print `name ₹price`.
- `addItem(conn, name, priceInr, stall)` — `INSERT` with `?` only; print `Inserted rows:` + count.

In `main`: connect as `root`, `listByStall "South"`, `addItem("Filter coffee", 25, "South")`, `listByStall "South"` again. Catch `SQLException` with a clear message. No `Statement`. No Spring.

```
// your code
```

2. Write a **CampusCart / TechMart canteen order** HTML page (no CSS, no JS). Semantic `header` / `main` / `footer`. One `h1`. A **POST** form (`action="/canteen/order"`) with labeled fields: student name, item, quantity. Each `<label for>` matches the input `id` and `name`. Include **one** accessible group: `fieldset` + `legend` with radio **UPI** / **Cash**. Submit with `<button type="submit">`. No Flex, no media queries, no `fetch`.

```
<!-- your code -->
```

C. Short answer (1)

In 3–6 sentences, explain **INNER JOIN vs LEFT JOIN** for a campus library (`members` and `loans`) or **AcmeHR** (`employee` and `leave_record`): who disappears, what NULL means, and why Java still binds the id with `PreparedStatement` `?` (not string-concat SQL). You may name the `ON` columns; you do **not** need a full three-table query.

(Write 3–6 sentences.)